TRABAJO PRÁCTICO 1

PROBLEMA 1 – DIVISIÓN Y CONQUISTA

Se tiene una bolsa con n monedas de idéntica denominación, de las cuales exactamente una es falsa y se sabe que es más liviana que el resto.(Es la única forma de diferenciarla)

- Diseñar e implementar un algoritmo de División y Conquista que identifique la moneda falsa, sin recurrir a ordenar las monedas por su peso.
- Calcular el orden de complejidad temporal del algoritmo desarrollado.
- Utilizar tamaños de datos con n={100, 500, 1000, 5000, 10000, 50000, etc.}
- Graficar el tiempo de ejecución y la cantidad de pesadas realizadas en cada caso y relacionarlos con el orden de complejidad.
- ¿Es correcto vincular el tiempo de ejecución con la cantidad de pesadas? ¿Por qué?

PROBLEMA 2 – GREEDY

Un equipo de antropología encontró un yacimiento con un código extraño escrito en un lenguaje incomprensible. No están seguros, pero algunos miembros del equipo consideran que se trata de un algoritmo que trabaja sobre grafos. Lo que les pedimos es que resuelvan los siguientes interrogantes:

- ¿Qué problema resuelve el código encontrado?
- Algoritmo. ¿Qué algoritmo conocido replica dicho código?
- Estructuras de datos. ¿Cuál es el uso de las principales estructuras de datos del programa?
- ¿Por qué se puede considerar que es un algoritmo Geedy? ¿Cuál es la regla que se aplica?
- Complejidad temporal y espacial. ¿Cuál es el orden de complejidad temporal y espacial del programa? NOTA: no se requiere graficar, sólo una interpretación teórica.
- Traducción. Replicar el código en Python lo más literalmente posible.
- Como datos de prueba, se puede usar la siguiente tabla:

Eje	Costo
1,2	66
2,3	122
2,4	126
3,4	80
3,5	148
4,5	126

5,6	49
6,7	101
7,8	69
7,9	72
8,9	45
8,11	56
11,10	30
10,9	46

PROBLEMA 3 – BACKTRACKING

Paolo Casanova ha mejorado su aspiradora robot: ahora puede resolver laberintos. Para ello, la aspiradora puede realizar tres operaciones:

- Ver lo que tiene adelante: Pared, camino ó la salida
- Avanzar una posición
- Girar 90º a la izquierda.

Con esas operaciones, le alcanza para recorrer laberintos rectangulares, que estén codificados de forma matricial como la siguiente tabla:

X	X	X	X	X	X	X	X
X					X		S
X		X	X	X	X		X
X				X	X		X
X		X					X
X	E	X	X	X	X	X	X

Donde “X” significa Pared, el espacio en blanco es el camino, “S” es la salida” y “E” es la posición inicial de la aspiradora.

- Diseñar, implementar y documentar un algoritmo mediante backtracking que dado un laberinto de tamaño mxn, encuentre el camino a seguir para salir del mismo.
- Supuestos: identificar supuestos, condiciones o premisas bajo los cuales funcionará el algoritmo
- Diseño: Pseudocódigo y estructuras de datos utilizadas
- Seguimiento: Ejemplo de seguimiento con un set de datos reducido
- Complejidad: Análisis de la complejidad temporal a partir del pseudocódigo
- Sets de datos: diseñar sets de datos aleatorios del tamaño solicitado en cada caso. Se pueden generar utilizando una planilla de cálculo o cualquier herramienta que se considere adecuada. Cada set de datos debe ser incluido en la entrega, junto con el resultado obtenido en cada caso.

- Tiempos de Ejecución: medir los tiempos de ejecución de cada set de datos y presentarlos en un gráfico.
- Informe de Resultados: redactar un informe de resultados comparando los tiempos de ejecución. ¿Se corresponde con la complejidad determinada inicialmente?

PROBLEMA 4 – PROGRAMACIÓN DINÁMICA

Cualquier cadena puede ser descompuesta en secuencias de palíndromos. Por ejemplo, la cadena ARACALACANA se puede descomponer de las siguientes formas:

ARA CALAC ANA
ARA C ALA C ANA
A R A CALAC A N A
etc.
(en el peor de los casos, un texto de longitud n se dividirá en n cadenas de longitud 1)

Desarrollar un algoritmo de programación dinámica que encuentre el menor número de palíndromos que forman una cadena dada. Por ejemplo, para ARACALACANA debería devolver 3.

Se pide:

1. Supuestos: identificar supuestos, condiciones, limitaciones y/o premisas bajo los cuales funcionará el algoritmo desarrollado
2. Diseño:
    a. Definir la Ecuación de Recurrencia
    b. Justificar el cumplimiento de los requisitos de Subestructura Óptima y Subproblemas Superpuestos
    c. Indicar cómo usa Memoization
    d. Incluir un Pseudocódigo
    e. Detallar las estructuras de datos utilizadas
3. Seguimiento: Ejemplo de seguimiento con un set de datos reducido
4. Complejidad: Análisis de la complejidad temporal a partir del pseudocódigo
5. Sets de datos: diseñar sets de datos apropiados. Se pueden generar utilizando una función random con una semilla fija, para permitir la reproducibilidad de los resultados, o ser generados manualmente e incluidos como archivos que lee el programa.

. Cada set de datos debe ser incluido en la entrega, junto con el resultado obtenido en cada caso. El programa entregado debe generar los sets de datos o leerlos desde archivos.

6. Tiempos de Ejecución: medir los tiempos de ejecución de cada set de datos y presentarlos en un gráfico.
7. Informe de Resultados: redactar un informe de resultados comparando los tiempos de ejecución. ¿Se corresponde con la complejidad determinada inicialmente?

. El gráfico comparativo de tiempos debe incluir tanto la curva con los valores medidos como la curva teórica de ajuste.

Condiciones Generales de Entrega

- El trabajo debe ser entregado en un archivo zip conteniendo:
  - Documento con carátula, índice y numeración de páginas. La carátula debe incluir nombre y padrón de los integrantes del grupo. Debe presentarse en formato PDF.
  - Archivos con los fuentes desarrollados. Indicar en el documento el lenguaje de programación utilizado e incluir instrucciones para compilar (de ser necesario) y ejecutar.
  - Archivos con los sets de datos utilizados
  - Archivos con resultados obtenidos para cada set de datos
- Si se incluyeran referencias bibliográficas, utilizar normas APA 7ma edición

ANEXO PROBLEMA 2

Archivo “paolo.bas”

```
10 CLS
20 INPUT "INGRESE EL NUMERO DE VERTICES "; N
30 PRINT
40 DIM COST (N,N)
50 DIM DIST (N)
60 DIM SOL (N)
70 A=1
80 PRINT "INGRESE EL CUADRO DE COSTOS (INGRESE 0,0 PARA TERMINAR)"
90 PRINT
100 INPUT "EL EJE "; A, B
110 IF A=0 THEN 140
120 INPUT "COSTO DEL EJE "; COST(A,B)
130 GOTO 100
140 FOR I=1 TO N
150 FOR J=1 TO N
160 IF COST (I,J)=0 THEN COST (I,J)=15000
170 NEXT J
180 NEXT I
190 FOR I=1 TO N
200 FOR J=1 TO I
210 COST (I,J)=COST (J,I)
220 NEXT J
230 NEXT I
240 INPUT "INGRESE EL VERTICE DE SALIDA "; V
250 FOR I=1 TO N
260 DIST(I)=COST(V,I)
270 SOL(I)=0
280 NEXT I
290 GOSUB 1000
300 PRINT "SALIDA","LLEGADA","DISTANCIA"
310 FOR I=1 TO N
320 IF DIST(I)<15000 THEN PRINT V, I, DIST(I)
330 NEXT I
340 INPUT "OTRA VEZ? (SI/NO) "; RES$
350 IF RES$="NO" THEN END
```

```
360 FOR I=1 TO N
370 SOL(I)=0: DIST(I)=0
380 NEXT I
390 GOTO 240
1000 SOL(V)=1
1010 DIST(V)=0
1020 FOR I=1 TO N-1
1030 U=15000
1040 FOR J=1 TO N
1050 IF DIST(J)<=U AND SOL(J)=0 THEN U=J
1060 NEXT J
1070 SOL(U)=1
1080 FOR J=1 TO N
1090 IF DIST(J)>= (DIST(U)+COST(U,J)) THEN DIST(J)=DIST(U)+COST(U,J)
1100 NEXT J
1110 NEXT I
1120 RETURN
```